1. CONCURRENCY CONTROL, DENORMALIZATION & DATA REPLICATION (DBMS)

1. CONCURRENCY CONTROL - OVERVIEW

- Concurrency allows multiple transactions to execute at the same time (overlap).
- Concurrency control ensures these concurrent transactions do not interfere with each other and maintain data consistency and isolation.
- Goal : Maximize safe concurrency while preventing anomalies.
- Without concurrency control : one transaction may overwrite another, read uncommitted data, or produce inconsistent results.

2. COMMON CONCURRENCY PROBLEMS

Problem	What Happens ?	Example
Dirty Read	Read data from uncommitted transaction which may be rolled back.	T2 reads value updated by T1 (not committed).
Lost Update	Two transactions update same data, one update gets lost/overwritten.	T1 and T2 update balance, last write wins.
Non-repeatable Read	Same query gives different result on re-execution within a transaction.	T1 reads 500, later reads 600 (committed by T2).
Phantom Read	New rows appear in repeated query.	T1 counts 10 rows, later 12 after T2 inserts new rows.
Write Skew	Two transactions update different rows violating a business rule.	Two doctors on-call both go off, constraint violated.

3. CONCURRENCY CONTROL TECHNIQUES

Technique	How it works ?	Key Points	Pros	Cons
Lock Based (2PL)	Transactions acquire locks (shared/read, exclusive/write).	• Two Phase Locking : 1. Growing (acquire locks) 2. Shrinking (release locks)	• Prevents conflicts, ensures serializability.	• Blocking, deadlocks, reduced concurrency.
Timestamp Ordering	Each transaction gets a timestamp. Operations follow a valid order.	• If conflict occurs, transaction is aborted and retried.	• No deadlocks, maintains order.	• More rollbacks/ retries under high conflicts.
Optimistic Concurrency	Transactions execute without locking, validate at commit.	• Check for conflicts before commit (versioning).	• Good when conflicts are rare, high concurrency.	• Retries if conflicts are frequent.
MVCC (Multi Version (Concurrency Control)	Maintains multiple versions of data for readers.	• Readers see a consistent snapshot, writers create new versions.	• Non-blocking reads, better read performance.	• Requires version management, storage overhead.

4. LOCK TYPES (LOCK BASED)

Lock Type	Description
Shared Lock (S)	Allows multiple transactions to read but not modify.
Exclusive Lock (X)	Allows one transaction to write. No other read or write allowed.
Intent Lock	Indicates intention to acquire lock on lower level (used in hierarchical locking).
Granularity	Locks can be at row, page, table or database level.

5. SCHEDULES (BASED ON RECOVERY / ISOLATION)

Schedule Type	Meaning
Recoverable	If T2 reads data written by T1, T2 cannot commit before T1 commits.
Cascadeless	Transactions cannot read uncommitted data of other transactions.
Strict	Transactions cannot read or overwrite data written by another transaction until it commits or aborts.
Strength Order	Strict → Cascadeless → Recoverable

6. ACID WITH CONCURRENCY (QUICK RECALL)

Property	Role in Concurrency Control
Atomicity	Ensures transaction is all or nothing (rollback on failure).
Consistency	Maintains valid state (constraints).
Isolation	Controls concurrent transactions from interfering.
Durability	Committed changes are permanent even if crash occurs.

7. CONCURRENCY - ADVANTAGES & DISADVANTAGES

Advantages

- Allows multiple transactions
- Improves performance and throughput
- Better utilization of resources
- Maintains data integrity
- Supports scalability

Disadvantages

- Lock/validation overhead
- Possible deadlocks
- More retries/aborts
- Long transactions cause contention
- Does not guarantee zero conflicts in all isolation levels

2. DENORMALIZATION & DATA REPLICATION (DBMS)

1. DENORMALIZATION - OVERVIEW

- Denormalization is the process of intentionally introducing controlled redundancy or precomputed data to improve query performance and simplify analytical access.
- It is an optimization technique applied after normalization. It is not the opposite of normalization.
- Common in read-heavy analytical systems (Data Warehouse, BI).
- Helps reduce complex joins and recalculations.

2. NORMALIZED vs DENORMALIZED (EXAMPLE)

Normalized Structure



Denormalized Structure (Analytical Table)

order_id	product_name	city	sales
0101	Laptop	Delhi	50,000
0102	Mouse	Pune	1,000

Frequently used attributes in single table to improve query performance.

3. METHODS OF DENORMALIZATION

Technique	Description	Use Case
Combining Tables	Store related attributes in same table.	Customer city with sales data.
Repeating Attributes	Duplicate frequently used attributes.	Product category in fact table.
Precomputed Aggregates	Store aggregated values (e.g., daily sales).	Faster dashboard queries.
Materialized Views	Persist query results.	Complex join / heavy queries.
Derived Columns	Store calculated values (e.g., net revenue).	Avoid repeated calculations.

4. DENORMALIZATION - PROS & CONS

Advantages

- Fewer joins, simpler queries.
- Faster read/query performance.
- Can reduce repeated calculations.
- Useful for analytical workloads.

Disadvantages

- Data redundancy and inconsistency risk.
- More storage required.
- More complex to maintain and update.
- Not suitable for high transactional workloads.

5. NORMALIZATION vs DENORMALIZATION

Aspect	Normalization	Denormalization
Redundant Data	Removed	Introduced (controlled)
Tables	More tables (normalized)	Fewer tables / wider tables
Query Performance	More joins, can be slower	Faster, simpler queries
Data Integrity	High consistency	Risk of inconsistencies
Use Case	OLTP, transactional systems	OLAP, Data Warehouse, reporting
Update Frequency	Frequent updates	Read-heavy, infrequent updates
Storage	Less storage	More storage

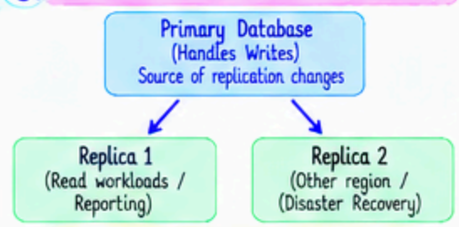
6. DATA REPLICATION - OVERVIEW

- Data replication is the process of maintaining a copy of data across multiple databases/servers/locations.
- It improves read performance, availability, scalability and fault tolerance.
- Replicas may not always be identical at the same instant (replication lag).
- Used in OLTP systems, analytical systems, cloud and multi-region setups.

7. TYPES OF REPLICATION

Type	How it works ?	Use Case
Snapshot	Copy data at a particular point in time.	Reference/slowly changing data.
Transactional	Replicates committed changes as they occur.	Frequent ongoing changes (OLTP).
Merge	Multiple copies independently updated and later synchronized (conflict resolution).	Offline systems, distributed apps.
Topology Types	Primary-Replica, Multi-Primary, Peer-to-Peer.	Depends on availability and write requirements.
Based on Sync	Synchronous (waits for replica). Asynchronous (faster, lag possible).	Sync for strong consistency, Async for lower latency.

8. REPLICATION ARCHITECTURE (EXAMPLE)



Actual configuration depends on database and requirements (Sync/Async, Single or Multi-primary etc.).

9. ADVANTAGES & DISADVANTAGES OF REPLICATION

Advantages

- Faster read performance.
- Improved availability and fault tolerance.
- Distributes read workload.
- Supports geographical access.
- Helps in disaster recovery.
- Reduces read load on primary.

Disadvantages

- More storage and network cost.
- Replication lag (data may be behind).
- Maintenance and monitoring overhead.
- Conflict resolution in multi-writer setups.
- Accidental deletions can also be replicated.
- Does not replace proper backups.

10. KEY CONSIDERATIONS

- Monitor replication lag and data freshness.
- Track source and replica positions/offsets.
- Handle failover and recovery process.
- Ensure schema compatibility across replicas.
- Use CDC for incremental ingestion in warehouses.
- Replication is not a backup strategy.
- Use idempotent processing and reconciliation for consistency.

2. SCENARIO-BASED INTERVIEW QUESTIONS (CONCURRENCY, DENORMALIZATION & DATA REPLICATION)

1. PREVENT OVERSLLING IN INVENTORY (CONCURRENCY CONTROL)

Problem / Scenario

- Two users try to purchase the last item
- in inventory at the same time.
  How do you prevent overselling?

Expected Answer / Approach

- Use transaction with atomic update.
- Use locking or optimistic concurrency.
- Update stock only if available (stock > 0).
- Check affected rows to confirm success.
- Handle failure gracefully (retry or show out of stock).

Example (SQL)

```
BEGIN TRANSACTION;
UPDATE inventory
SET stock = stock - 1
WHERE product_id = 101
  AND stock > 0;

-- Check if 1 row updated
-- If yes, create order
-- If no, rollback (out of stock)
COMMIT;
```

How it works ?

1. Both transactions try to update.
2. DB uses locks/row level locking.
3. Only one transaction succeeds (stock > 0), other gets 0 rows updated.
4. Ensures atomicity and prevents overselling.

Before / After Example

product_id	stock	Transaction	Result
101	1	T1 updates	Success (stock=0)
101	0	T2 updates	Fails (0 rows updated)

Follow-up Questions

- Why is SELECT stock followed by UPDATE not sufficient ?
  → Race condition; another transaction can modify in between.
- What lock type is used ?
  → Row level exclusive lock (implementation dependent).
- How to handle high contention ?
  → Use optimistic concurrency/retry mechanism.

2. TWO ETL JOBS UPDATE SAME CUSTOMER RECORD (CONCURRENCY)

Problem / Scenario

- Two ETL jobs run in parallel and update
- the same customer record. One job's
- update overwrites the other's changes.
  How do you handle this?

Expected Answer / Approach

- Use optimistic concurrency (version column) or locking mechanism.
- Detect conflicts and retry.
- Serialize updates for the same key.
- Ensure idempotent processing in pipelines.

Example (Optimistic Locking)

customer_id	name	city	version
C01	Rahul	Delhi	3

```
UPDATE customers
SET city = 'Pune', version = version + 1
WHERE customer_id = 'C01' AND version = 3;
```

If 0 rows updated → another job already updated, so fetch latest data and retry.

Timeline (Without Concurrency Control)

Time	Job A	Job B	Result
T1	Reads city = Delhi		
T2		Reads city = Delhi	
T3	Updates city = Pune		Pune
T4		Updates city = Mumbai	Mumbai (Overwrites A)

Timeline (With Optimistic Locking)

Time	Job A	Job B	Result
T1	Reads city = Delhi (v=3)		
T2		Reads city = Delhi (v=3)	
T3	Updates to Pune (v=4)		Success
T4		Tries update (v=3)	Fails (0 rows updated)

Follow-up Questions

- Is MVCC enough to avoid write conflicts? → No, write conflicts still need detection/retry.
- When to use pessimistic locking? → High contention, critical data (e.g., payments).
- How to ensure idempotency in ETL? → Use merge/upsert with business keys and source timestamp.

3. DASHBOARD WITH 12 TABLE JOINS TAKES 40 SECONDS (DENORMALIZATION)

Problem / Scenario

A reporting dashboard joins 12 normalized tables and takes 40 seconds to load.
Would you denormalize the data?
If yes, how and what are the considerations?

Expected Answer / Approach

- First analyze query execution plan, indexes, filters and data volume.
- If frequently used query, consider denormalization.
- Options: wide table, materialized view or pre-aggregated summary table.
- Measure performance before and after.
- Ensure data is refreshed and consistent.

Before (Normalized)



Complex joins → slower query

After (Denormalized/Analytical Table)

order_id	customer_name	product_name	store_city	sales	order_date
0101	Rahul	Laptop	Delhi	50000	2025-01-01
0102	Priya	Mouse	Pune	1000	2025-01-02

Frequently used attributes in single table → faster and simpler query

Methods

1. Combine tables (store attributes together)
2. Repeating attributes (e.g., customer city in orders)
3. Precomputed aggregates (daily sales, monthly revenue)
4. Materialized views (persist query results)
5. Derived columns (e.g., net_revenue = price - discount)

Pros

- Fewer joins, simpler queries
- Faster dashboard/query performance
- Can reduce repeated calculations
- Useful for analytical workloads

Cons

- Data redundancy and inconsistency risk
- More storage
- Need refresh/maintenance logic
- Not suitable for high transactional workloads

4. READ REPLICA SHOWS 950 ORDERS, PRIMARY SHOWS 1,000 (REPLICATION LAG)

Problem / Scenario

An e-commerce system uses a read replica for reporting. The primary has 1,000 orders but the replica shows only 950 orders.
What would you do?

Expected Answer / Approach

- This is replication lag (asynchronous).
- Check replica lag and replication health.
- Verify whether missing 50 orders are pending replication or actual data issue.
- Monitor replication metrics (LSN/offset).
- If real-time consistency is required, route query to primary or wait for lag to catch up. Implement alerting for lag beyond SLA.

Architecture



Example - Checking Replication Lag

PostgreSQL	MySQL
SELECT * FROM pg_stat_replication;	SHOW SLAVE STATUS;

primary_orders	replica_orders	lag	status
1000	950	50	CATCHING_UP

Follow-up Questions

- Can we assume latest committed write is instantly visible on replica? → No, not in async replication.
- How to handle if dashboard needs near real-time data? → Use primary or wait for lag, or use sync replication if required.
- What can cause replication lag? → High write load, network latency, large transactions, slow replica, resource constraints.

5. PRIMARY DATABASE CRASHES - CAN WE GUARANTEE ZERO DATA LOSS ? (REPLICATION)

Problem / Scenario

Primary database crashes and we have an asynchronous replica. Can we guarantee zero data loss? What steps will you take for recovery?

Expected Answer / Approach

- No, we cannot guarantee zero data loss in asynchronous replication.
- Some committed transactions may not have reached the replica.
- Check replication position / lag.
- Use backups + replica for recovery.
- Understand RPO (data loss tolerance) and RTO (recovery time).
- After failover, verify data consistency.

Failover Architecture



Steps for Recovery

1. Check last committed transaction on replica (replication position).
2. Promote replica to primary.
3. If data loss, restore from backup and reapply missing transactions.
4. Verify data consistency.
5. Redirect application to new primary.

Considerations

- Asynchronous replication may lose recent commits.
- Use backups for full recovery (Point-in-Time Recovery).
- Test failover process regularly.
- Ensure no split-brain (old primary coming back online).

6. OUT-OF-ORDER CDC EVENTS CAUSE INCORRECT FINAL STATUS (REPLICATION + CDC)

Problem / Scenario

CDC events for an order arrive out of order:
INSERT (100) → UPDATE (102) → UPDATE (101).
If we process in arrival order, final status becomes SHIPPED instead of DELIVERED.
How do you ensure latest source change wins?

Expected Answer / Approach

- Use source sequence number / timestamp (LSN) to maintain order.
- Store and process events based on sequence, not arrival time.
- Use MERGE/UPSERT logic with version condition.
- Keep last processed sequence for each key.
- Ensure idempotent processing (handle retries, duplicates).
- Optionally buffer events for a small time window if out-of-order arrival is common.

Events Received (Out of Order)

Arrival Order	Order ID	Status	Source Sequence
1	0101	CREATED	100
2	0101	DELIVERED	102
3	0101	SHIPPED	101

MERGE / UPSERT Example (using sequence)

```
MERGE INTO orders t
USING staging_events s
ON t.order_id = s.order_id
WHEN MATCHED AND s.source_seq > t.source_seq THEN
  UPDATE SET status = s.status, source_seq = s.source_seq,
WHEN NOT MATCHED THEN
  INSERT (order_id, status, source_seq)
  VALUES (s.order_id, s.status, s.source_seq);
```

Key Points

- Maintain source_seq / LSN / event timestamp.
- Always apply latest sequence.
- Handle duplicates (idempotent).
- Use watermarking or buffering for high out-of-order rate.
- Ensure correct delete handling.
- Monitor for missing sequence numbers.